Building an App

The main parts of an app, what good looks like in each, and the skill that builds it and the skill that checks it. Work through them top to bottom.

How to use it: install the skills, open your app in Claude Code and say *"Read the app playbook and take me through Design"* (or whichever part you are on).

PART	BUILD WITH	CHECK WITH
Setup	`setup-new-repo`	`audit-documentation`
Design	`setup-design-system`	`audit-design-system`
Data and auth	`supabase`	`audit-database`
APIs	`setup-error-handling`	`audit-api`
Error handling	`setup-error-handling`	`audit-error-handling`
Loading		`audit-page-loading`
Testing	`setup-testing`	`audit-tests`
Security		`audit-security`
Monitoring	`setup-sentry`	`audit-observability`
Launch		`audit-production-readiness`

Build skills add something to your app. Check skills score it and list what to fix. Run one at a time: each reads what the last one left behind.

Setup

Before any code. Gives the agent the rules of your repo and one command that proves work is done.

- `AGENTS.md` holds only what the agent cannot work out from the code: commands, traps, decisions.
- One `verify` command runs types, lint, tests and build. Green means done.
- CI runs `verify` on every pull request.
- Deploy a first page early, so every change ships the same way.

BUILD `setup-new-repo` CHECK `audit-documentation`

✓ DONE WHEN `verify` passes in CI and a first page is live on a preview link.

Design

Before you build screens. Changing the look later costs far more.

- Pick one look for the whole app.
- Colours, spacing and corner radius come from design tokens, never hard-coded values.
- Build from shared components (button, card, input). Never hand-roll one.
- Write the design docs from the real code once tokens and components exist.

BUILD `setup-design-system` CHECK `audit-design-system`

✓ DONE WHEN a real page uses only tokens and shared components.

Data and auth

Before features. The database is the hardest thing to change later.

- Row-level security on every table, so the database itself blocks the wrong user.
- Logging in and permission are different things. Decide, table by table, who can read and write.
- Check permission on the server for every request, not just in the page.
- Change the database only through migrations. Index what you search and sort on.

BUILD `supabase` CHECK `audit-database`

✓ DONE WHEN every table has row-level security and a test proving another user is blocked.

APIs

Every route and server action.

- Validate every input before using it.
- Check the user's permission inside every action.
- One error format across every route.
- Timeouts and retries on calls to outside services.

BUILD `setup-error-handling` CHECK `audit-api`

✓ DONE WHEN every route validates input, checks permission and has a test.

Error handling

From the first API route.

- Expected errors get a clear message and a retry button.
- Unexpected errors land on a friendly fallback screen, never a crash or a stack trace.
- Never hide an error silently.
- When something breaks, deny access. Never grant it by accident.

BUILD `setup-error-handling` CHECK `audit-error-handling`

✓ DONE WHEN a forced error on each page shows a friendly screen and gets logged.

Loading

Once real pages exist, and again before launch.

- Measure real users first, then lab tools.
- Skeletons while content loads, spinners for button actions.
- Load data in parallel, not one request after another.
- The app frame and navigation appear instantly.

CHECK `audit-page-loading`

✓ DONE WHEN key pages pass Google's Core Web Vitals for real users.

Testing

Set it up at your first feature. Every feature after adds tests.

- Write the failing test first, then the code.
- End-to-end tests for sign-in, payments and the main feature.
- Every bug becomes a test, so it cannot come back.
- Check each test fails when the code is wrong. Agents write tests that always pass.

BUILD `setup-testing` CHECK `audit-tests`

✓ DONE WHEN all tests run in CI and the main user journeys are covered end to end.

Security

Before launch, then monthly.

- Secrets live in environment variables, never in code the browser sees.
- Limit how often people can try to sign in or hit expensive routes.
- Security headers on every page.
- Business-logic bugs are security bugs, such as a free user getting paid features when something breaks.

CHECK `audit-security`

✓ DONE WHEN the audit is clean against the OWASP Top 10.

Monitoring

Before anyone else uses the app.

- Errors are tracked and point at your real code.
- Logs show which request and which user.
- You get an alert on a new error, before users tell you.

BUILD `setup-sentry` CHECK `audit-observability`

✓ DONE WHEN a test error in production reaches Sentry and sends you an alert.

Launch

Before the first real users.

- Every change goes to a preview link first, then to production.
- Database changes ship before the code that needs them.
- Know how to roll back, and practise it once.
- Backups on, and one restore tested.
- Accessibility, page titles and share images, privacy policy.
- Email set up so it does not land in spam.
- Spending alerts on hosting and paid services.

CHECK `audit-production-readiness`

✓ DONE WHEN you have rolled back once and restored a backup once.

Getting the skills

Install everything in this playbook:

```
npx skills add stevetowers08/app-template-kit -g -a claude-code
```

Two come from their makers: `npx skills add shadcn/ui` for components and `npx skills add supabase/agent-skills` for the database.

Two more are included: `research` checks current best practice on any topic, and `update-skills` keeps these skills up to date.

Sources

Next.js production checklist · Next.js data security · Supabase production checklist · Vercel production checklist · Core Web Vitals · OWASP Top 10 · Google SRE: reliable launches · Claude Code best practices · agents.md